

Algorithmic Game Theory Project: Network Security Games

Di Franco Federico
Serratore Francesco
Visciglia Domenico

January 19, 2026

Contents

1	Introduction	2
2	Task 1: Strategic Game Modeling	2
2.1	Game Formulation	2
2.2	Theoretical Validation of Equilibria	3
2.3	Iterative Equilibrium Finding Algorithms	3
2.4	Large-Scale Scalability: Sparse Matrix & JIT Optimization	4
3	Task 2: Coalitional Game Theory	6
3.1	Characteristic Function	6
3.2	Shapley Value Approximation	6
3.3	Set Construction: Reverse Greedy Heuristic	7
4	Task 3: Market Allocation	7
4.1	Market Model	7
4.2	Allocation Mechanisms	8
5	Task 4: VCG Path Auction	9
5.1	Mechanism Design	9
5.2	VCG Payment Rule	10
5.3	Visual Analysis	10
6	Bonus Section: Exact Benchmark via SAT Solver	11
6.1	Symbolic Encoding	11
6.2	Cardinality Constraint: The Sequential Counter	11
6.3	Optimization Strategy	11
7	Conclusion and Comparative Analysis	12
7.1	Performance Benchmark	12

1 Introduction

The primary objective of this project is to address the **Network Security problem**: securing a graph $G = (V, E)$ such that the traffic on every edge is controlled by at least one secure node. According to the project specification, a set of nodes $S \subseteq V$ is defined as a *Network Security Set* if, for every node $i \in V$, either $i \in S$ or all its neighbors $N(i)$ are in S ($N(i) \subseteq S$). We observe that this condition implies that every edge in the graph must be incident to at least one node in S , making the problem topologically equivalent to the **Minimal Vertex Cover** (MVC) problem, which is known to be NP-hard.

We approach this optimization problem through the lens of Algorithmic Game Theory, decomposing the analysis into four distinct computational tasks:

1. **Strategic Game Modeling:** Defining a non-cooperative game where Pure Nash Equilibria (PNE) correspond to valid security sets.
2. **Coalitional Analysis:** Using Cooperative Game Theory and Shapley Values to assess the marginal importance of each node in maintaining network security.
3. **Market Allocation:** Simulating an economic environment where secure nodes (Buyers) purchase services from Vendors subject to budget and capacity constraints.
4. **Mechanism Design:** Implementing a truthful VCG (Vickrey-Clarke-Groves) auction to procure secure paths within the network.

Exact Benchmark (Bonus Analysis). As a bonus implementation version, we went beyond the core requirements and implemented an exact solver based on Boolean Satisfiability (SAT). This approach is inherently limited by the NP-hard nature of the problem and does not scale to large networks. All experiments are conducted on three distinct network topologies: Regular, Erdős-Rényi, and Barabasi-Albert graphs.

2 Task 1: Strategic Game Modeling

We modeled the Network Security problem as a strategic game $\Gamma = \langle N, \{S_i\}, \{u_i\} \rangle$ where nodes act as selfish agents aiming to minimize their cost while ensuring security.

2.1 Game Formulation

The game environment is implemented in the class `SecurityGame`.

- **Players (N):** The set of nodes V in the graph.
- **Strategies (S_i):** Each player i selects a strategy $s_i \in \{0, 1\}$.
 - $s_i = 1$: The node secures itself (incurring a cost c).
 - $s_i = 0$: The node does not secure itself (relying on neighbors).
- **Payoff Function (u_i):** The utility function logic incentivizes "Free-Riding" only if the node is safely covered by at least one neighbor. Let $N(i)$ be the set of neighbors

of i . The payoff is:

$$u_i(s) = \begin{cases} \alpha - c & \text{if } s_i = 1 \\ \alpha & \text{if } s_i = 0 \wedge \forall j \in N(i) : s_j = 1 \\ 0 & \text{if } s_i = 0 \wedge \exists j \in N(i) : s_j = 0 \end{cases} \quad (1)$$

We utilized parameters $\alpha = 10$ (Value of Security) and $c = 4$ (Cost of Implementation). Since $\alpha > c > 0$, the game structure is a *Game of Strategic Substitutes*.

2.2 Theoretical Validation of Equilibria

Before implementing the learning dynamics, we rigorously verified that the stable states of this game correspond exactly to the problem objectives. We analyzed the incentives for a node i in four distinct scenarios:

Scenario	Strategy (s_i)	Is Covered?	Payoff	Incentive
Uncovered	0	No	0	Switch to 1 (Gain $\alpha - c$)
Redundant	1	Yes (by neighbors)	$\alpha - c$	Switch to 0 (Gain $\alpha > \alpha - c$)
Essential	1	No (neighbors=0)	$\alpha - c$	Stay 1 (Switching yields 0)
Free Rider	0	Yes (by neighbors)	α	Stay 0 (Switching costs c)

Table 1: Strategic incentives analysis confirming stability conditions.

Theorem: A strategy profile S is a Pure Nash Equilibrium (PNE) if and only if the set of secure nodes corresponds to a *Minimal Network Security Set*.

Proof.

1. **Validity (Must cover all edges):** Suppose S is an equilibrium but not a valid security set. This implies $\exists i$ such that $s_i = 0$ and $\forall j \in N(i), s_j = 0$ (Uncovered scenario). The current payoff is 0. By switching to $s_i = 1$, the player gains $\alpha - c$. Since $\alpha > c$, this deviation is profitable. Thus, in equilibrium, no node remains uncovered.
2. **Minimality (No redundancy):** Suppose S is a valid security set but not minimal. This implies $\exists i$ such that $s_i = 1$, but i is already covered by at least one neighbor j with $s_j = 1$ (Redundant scenario). The current payoff is $\alpha - c$. By switching to $s_i = 0$, the player becomes a Free Rider and receives α . Since $\alpha > \alpha - c$, the player is incentivized to switch. Thus, in equilibrium, no redundant nodes exist.

2.3 Iterative Equilibrium Finding Algorithms

To find these equilibria, we implemented three learning dynamics.

Best Response Dynamics (BRD) Implemented in `BestResponseDynamics.py`.

- **Update Rule:** In each iteration, player i calculates the hypothetical payoff of switching strategies given the *current* profile of opponents s_{-i} and switches only if $u_{new} > u_{old}$.
- **Convergence:** The algorithm terminates when a full round passes with no strategy changes.

Fictitious Play (FP) Implemented in `FictitiousPlay.py`. Unlike BRD, players respond to the *historical frequency* of opponents' play.

- **Belief System:** Each node estimates the probability of a neighbor j playing '1' as $\hat{p}_j = \frac{\text{count}(j=1)}{\text{total_rounds}}$.
- **Expected Utility Derivation:** We derived the decision rule based on Expected Utility Theory. The probability that node i is covered by its neighbors is $P_{cov} = 1 - \prod_{j \in N(i)} (1 - \hat{p}_j)$. The expected utilities are:

$$E[U(s_i = 1)] = \alpha - c \quad (\text{Deterministic cost}) \quad (2)$$

$$E[U(s_i = 0)] = \alpha \cdot P_{cov} + 0 \cdot (1 - P_{cov}) = \alpha \cdot P_{cov} \quad (3)$$

- **Deterministic Threshold:** A rational agent plays $s_i = 0$ if and only if $E[U(0)] > E[U(1)]$:

$$\alpha \cdot P_{cov} > \alpha - c \implies P_{cov} > \frac{\alpha - c}{\alpha} \quad (4)$$

Using our parameters ($\alpha = 10, c = 4$), the node attempts to Free Ride only if the probability of being covered exceeds 0.6.

Regret Matching (RM) Implemented in `RegretMatching.py`.

- **Regret Calculation:** Player i compares the actual payoff with the counterfactual payoff $u(s'_i, s_{-i})$.
- **Update:** $R_i^{s'_i} \leftarrow R_i^{s'_i} + \max(0, u_{\text{counterfactual}} - u_{\text{actual}})$. Strategies are chosen probabilistically proportional to positive regret.
- **PNE Extraction:** A hard threshold ($p > 0.5 \implies 1$) is applied to the final distribution to extract a pure strategy.

2.4 Large-Scale Scalability: Sparse Matrix & JIT Optimization

While the initial NetworkX-based implementation served for prototyping, its object-oriented overhead and pointer-heavy structure make it unsuitable for massive networks. For instance, a standard dense adjacency matrix scales as $O(N^2)$, which for $N = 10^7$ would require approximately 100 terabytes of memory. To test the limits of *Best Response Dynamics* (BRD), we implemented a high-performance version in `ScalableBestResponse.py`.

Efficient Graph Generation: The Configuration Model. Generating a k -regular graph for tens of millions of nodes is a significant bottleneck. A naive linear approach (adding edges one by one while checking for constraints) would result in a complexity of at least $O(N \cdot k)$ with high constant factors due to repeated lookups. Instead, we adopted a **probabilistic Configuration Model** approach:

- **Stub Shuffling:** We represent each node as having k "stubs" (half-edges). We create an array of size $N \times k$ and use `numpy.random.shuffle` to randomize connections globally. This utilizes vectorized C-level shuffles, which are orders of magnitude faster than Python-level loops.

- **Probabilistic Validity:** The algorithm is probabilistic because random shuffling may generate *self-loops* or *multi-edges*. Rather than performing expensive per-edge validation, we utilize a "global check and retry" strategy. We validate the entire graph structure once it is generated; if it is not a "simple" graph, we restart the shuffle. For sparse graphs ($k \ll N$), the probability of generating a simple graph is high, making this approach extremely efficient.

Implementation Strategy and Performance. To achieve execution speeds comparable to C++ while maintaining the flexibility of Python, we combined three core technologies:

- **Sparse Matrices (CSR Format):** We utilized the **Compressed Sparse Row (CSR)** format via `scipy.sparse`. This reduces memory complexity to $O(N + E)$, storing only the non-zero entries (edges) and their offsets. This allows us to fit a graph with 10 million nodes and 30 million edges into a few hundred megabytes of RAM.
- **Numba JIT Compilation:** The core dynamics loop is decorated with `@njit`. Numba translates the Python logic into LLVM machine code. This is crucial for the *Best Response* calculation, as it allows the sequential update of nodes to run at native speed, effectively bypassing the Global Interpreter Lock (GIL).
- **Vectorization and Parallelism:** While the BRD is inherently sequential to ensure convergence, the graph generation and verification stages are heavily vectorized. By using NumPy arrays, we leverage SIMD (Single Instruction, Multiple Data) instructions, allowing the CPU to process multiple nodes simultaneously during the initial setup and payoff calculations.

As a result, the system can simulate a game with 10^7 players and reach convergence in a matter of seconds, a feat that would be impossible with traditional high-level graph libraries.

Benchmark Results. We conducted stress tests on random regular graphs ($K = 3$) of increasing size. The results (Table 2) demonstrate linear scalability, converging in just 3 iterations even for 50 million nodes.

Nodes (N)	Degree (K)	Iterations	Time (s)	Set Size (%)
5,000,000	3	3	0.7645	62.75%
10,000,000	3	3	3.9989	62.75%
20,000,000	3	3	5.0622	62.74%
50,000,000	3	3	7.5954	62.75%

Table 2: Performance benchmark of the JIT-optimized Best Response Dynamics.

3 Task 2: Coalitional Game Theory

In the second task, we shift our perspective from non-cooperative agents to a cooperative framework. Here, the focus is on assessing the "value" or importance of each node in forming a valid Network Security Set. To do this, we model the problem as a coalitional game and compute the **Shapley Value** for each node, which represents its average marginal contribution to the coalition's goal.

3.1 Characteristic Function

To define the coalitional game (N, v) , we constructed a characteristic function $v(S)$ that quantifies the security coverage provided by a coalition $S \subseteq N$. Based on the problem specification, the value of a coalition is the count of nodes that are "secure" according to the strict "ALL Rule":

$$v(S) = |\{i \in V \mid i \in S \vee N(i) \subseteq S\}| \quad (5)$$

In words, $v(S)$ represents the total number of secure nodes in the graph given that only the subset S is active. A node contributes to increasing this value either by securing itself (entering S) or by completing the security perimeter around a neighbor.

3.2 Shapley Value Approximation

The exact computation of Shapley values requires evaluating $2^{|N|}$ coalitions or summing over $|N|!$ permutations. Since this is computationally intractable for large networks, we implemented a **Monte Carlo approximation** in the class `CoalitionalSecurityGame`.

Marginal Contribution Logic We defined the marginal contribution of a node i to a coalition S based on the strict "ALL Rule" required by the Network Security Set definition. In our implementation, adding node i to a permutation generates a value of +1 if and only if it actively increases the security coverage of the network in one of two ways:

1. **Direct Contribution:** Node i itself becomes secured by entering the coalition (and it was not already secured by its neighbors).
2. **Indirect Contribution (Critical):** Node i completes the security requirement for a neighbor $j \notin S$.
 - According to the problem definition, a node j outside the set is secure only if *all* its neighbors are in the set.
 - Thus, node i generates value for j if and only if i is the **last missing neighbor** required to complete j 's coverage.

Algorithm Implementation The approximation algorithm works as follows:

- We generate a fixed number of random permutations of the players (parameterizable in the code).
- For each permutation, we scan nodes sequentially.

- We maintain a counter `current_neighbors_in_S` for every node to track how many of its neighbors have already appeared in the permutation.
- If adding node i increments this counter for neighbor j to match its degree ($\deg(j)$), and j is not in the set, we credit a contribution to i .

3.3 Set Construction: Reverse Greedy Heuristic

Once the Shapley values are computed, we use them as a ranking metric to construct a **Minimal Network Security Set**. Instead of a standard greedy approach (adding best nodes), which often fails to satisfy the strict "ALL neighbors" constraint efficiently, we implemented a **Reverse Greedy (Pruning)** approach:

1. **Initialization:** We start with the full set of nodes $S = V$ (which is trivially a secure set).
2. **Sorting:** We rank nodes by their Shapley values in **ascending order** (from least to most important).
3. **Pruning:** We iterate through the sorted list and attempt to remove each node i from S .
4. **Feasibility Check:** Node i can be removed if and only if its removal does not compromise the security of any neighbor.
 - Specifically, if a neighbor j is already excluded from the set ($j \notin S$), it relies entirely on its neighbors (including i) for security.
 - Therefore, if any neighbor of i is not in S , node i **cannot** be removed, as doing so would leave that neighbor uncovered.

This heuristic proved highly effective. In structured networks like Barabasi-Albert, the high-degree hubs naturally obtained high Shapley values (due to their potential to cover many leaves) and were correctly retained by the pruning process, while peripheral nodes were removed.

4 Task 3: Market Allocation

In this task, we simulate an economic environment where the nodes identified in the Security Set act as **Buyers** who must purchase security services from a set of **Vendors**. The goal is to maximize the aggregate Social Welfare of the system under different supply constraints.

4.1 Market Model

The market simulation is implemented in the class `SecurityMarketplace`.

- **Agents:**
 - **Buyers:** The subset of secure nodes found in previous tasks. Each buyer b is assigned a random budget $B_b \in [1, 100]$.

- **Vendors:** A set of service providers, each characterized by a price P_v , a security level $Q_v \in [1, 10]$, and a capacity K_v (maximum number of clients).
- **Utility Function:** We defined a utility function $U(b, v)$ that captures the trade-off between the quality of the service (security level) and the economic savings (budget surplus). To prioritize security quality over mere cost savings, we applied a weight factor (multiplier of 10) to the security level. The utility is defined as:

$$U(b, v) = \begin{cases} (Q_v \times 10) + (B_b - P_v) & \text{if } B_b \geq P_v \\ -\infty & \text{if } B_b < P_v \quad (\text{Incompatible}) \end{cases} \quad (6)$$

4.2 Allocation Mechanisms

We analyzed two distinct market scenarios, implementing specific algorithms for each to maximize welfare.

Scenario 1: Infinite Capacity (Unbounded Supply) In this scenario, vendors have infinite items ($K_v = \infty$). Since there is no rivalry for goods, the global optimum coincides with the local optimum for each agent.

- **Algorithm:** We implemented a **Local Greedy Search**. Each buyer independently iterates through all available vendors and selects the one that maximizes their individual utility $U(b, v)$.
- **Outcome:** This mechanism naturally leads to a Pareto-optimal allocation where the "Best Value" vendors (high quality/price ratio) capture the entire market share compatible with their price point.

Scenario 2: Limited Capacity (Scarcity) In this scenario, vendors have a finite capacity K_v . Simple local maximization is insufficient, as early movers could deplete high-value resources. To analyze the trade-off between computational speed and allocation efficiency, we implemented two distinct algorithms.

A. Heuristic Approach: Global Greedy We implemented a custom **Global Greedy** strategy aimed at approximating the maximum Social Welfare quickly.

1. **Sorting:** We generate all possible valid matches (b, v) and sort them in descending order based on utility $U(b, v)$.
2. **Assignment:** We iterate through the sorted list. A match is finalized if the buyer is unmatched and the vendor has remaining capacity.

While extremely fast ($O(M \log M)$), where M denotes the total number of potential matches (edges), defined as $N_{\text{buyers}} \times N_{\text{vendors}}$, this approach is not guaranteed to find the global optimum because a "good" local choice might block a "perfect" combination later.

B. Exact Approach: Min-Cost Max-Flow To find the mathematically optimal allocation, we modeled the market as a **Minimum Cost Maximum Flow** problem on a directed graph $G' = (V', E')$.

- **Nodes:** A source S , a sink T , all buyers B , and all vendors V .
- **Edges:**
 - $S \rightarrow b$ for each buyer, with Capacity = 1 and Cost = 0 (each buyer wants 1 item).
 - $v \rightarrow T$ for each vendor, with Capacity = K_v and Cost = 0 (vendor supply limit).
 - $b \rightarrow v$ if compatible, with Capacity = 1 and **Cost** = $-U(b, v)$.

Since standard network flow algorithms minimize total cost, setting the edge weight to the *negative utility* ensures that minimizing cost is equivalent to maximizing Social Welfare. We utilized the `networkx.max_flow_min_cost` algorithm to solve this flow network. The computational complexity of this approach is approximately $O(N \cdot M \cdot \log(V_{tot}))$, where N is the flow amount, M is the number of edges in the flow network, and $V_{tot} = N_{buyers} + N_{vendors} + 2$.

Performance Comparison. We benchmarked both approaches on the three network topologies. As shown in Table 3, the Greedy heuristic is orders of magnitude faster and provides solutions that are surprisingly close to (or even identical to) the global optimum.

Topology	Total Welfare		Execution Time (s)		Gap
	Greedy	Optimal	Greedy	Optimal	
Regular	9,290	9,592	0.0021	0.0756	3.2%
Erdős-Rényi	21,047	21,135	0.0010	0.0579	0.4%
Barabasi-Albert	11,920	11,920	0.0006	0.0219	0.0%

Table 3: Comparison between the Greedy Heuristic and the Exact Min-Cost Max-Flow algorithm. The Greedy approach achieves near-optimal welfare with significantly lower computational cost.

5 Task 4: VCG Path Auction

In the final task, we address the problem of procuring a secure path from a source node s to a target node t . We model this as a truthful mechanism design problem where the agents are the nodes themselves, each "selling" the right to traverse them.

5.1 Mechanism Design

The auction logic is encapsulated in the class `VCGPathAuction`. Since standard shortest path algorithms operate on edge weights, while our costs are associated with nodes, we implemented a specific graph transformation.

Graph Transformation and Cost Function To model node-based costs using standard Dijkstra algorithms, we constructed a temporary directed graph `temp_G`. For every undirected edge (u, v) in the original graph, we created two directed edges:

- $(u \rightarrow v)$ with weight w_v .
- $(v \rightarrow u)$ with weight w_u .

The "Social Cost" w_i of traversing node i is defined as the sum of its private cost (bid) and a public penalty representing the insecurity externality:

$$w_i = c_i + \text{Penalty}(i) \quad (7)$$

where $c_i \in [1, 20]$ is the private random bid, and $\text{Penalty}(i) = 10$ if $i \notin \text{Security Set}$ (0 otherwise). This effectively internalizes the negative externality of using insecure nodes into the path cost.

5.2 VCG Payment Rule

We implemented the Vickrey-Clarke-Groves (VCG) mechanism, which ensures truthfulness (dominant strategy incentive compatibility). The payment P_i for a winning node i on the optimal path is calculated as the externality it imposes on the system (the "harm" to social welfare if it were absent).

The formula used is:

$$P_i = \text{SP}(G \setminus \{i\}) - (\text{SP}(G) - w_i) \quad (8)$$

where $\text{SP}(G)$ denotes the total cost of the shortest path from s to t in graph G .

Implementation Details The method `run_vcg_mechanism` executes the following steps:

1. **Optimal Allocation:** Compute the shortest path on the full graph to identify the winning coalition of nodes.
2. **Payment Calculation:** For each winning node i (excluding source and target):
 - We create a temporary graph copy removing node i ($G \setminus \{i\}$).
 - We re-run the shortest path algorithm to find the best alternative solution ($\text{SP}(G \setminus \{i\})$).
 - **Monopoly Handling:** If removing node i disconnects s and t (making the target unreachable), the node is a "Bridge" with infinite monopoly power. In this case, the algorithm correctly assigns an infinite payment (or a very large cap), reflecting the lack of alternatives.

5.3 Visual Analysis

The results are visualized by color-coding the optimal path: green nodes represent secure transit points (low social cost), while red nodes indicate insecure points where the high penalty was paid because avoiding them would have been even more expensive (e.g., bottlenecks).

6 Bonus Section: Exact Benchmark via SAT Solver

In this bonus section, we deviate from the game-theoretic framework to explore a purely combinatorial approach. While the core of this project relies on decentralized agents self-organizing into Nash Equilibria, here we tackle the problem from a classic **Theoretical Computer Science** perspective.

6.1 Symbolic Encoding

The solver is implemented in `SatVertexCover.py` using the `SymPy` library for symbolic logic. We mapped the graph problem into a boolean formula Φ as follows:

- **Variables:** For each node $i \in V$, we defined a boolean variable x_i , where $x_i = \text{True}$ implies $i \in S$.
- **Edge Constraints:** To ensure validity, every edge $(u, v) \in E$ generates a clause $(x_u \vee x_v)$. The conjunction of all such clauses enforces the Vertex Cover property.

6.2 Cardinality Constraint: The Sequential Counter

The core challenge in SAT is enforcing the constraint "Choose at most k nodes" ($\sum x_i \leq k$), as boolean logic lacks arithmetic operators. Instead of naive encodings (which explode combinatorially), we implemented a **Sequential Counter** circuit.

We introduced auxiliary variables $S_{i,j}$ representing the state: "The sum of the first i variables is at least j ". The logic is defined recursively:

$$S_{i,j} \iff S_{i-1,j} \vee (S_{i-1,j-1} \wedge x_i) \quad (9)$$

This circuit effectively "counts" the active variables as we traverse the list of nodes. The final constraint $\neg S_{N,k+1}$ ensures that the total count never exceeds k .

6.3 Optimization Strategy

To find the *minimum* k :

1. We perform a **Binary Search** on the integer $k \in [0, |V|]$.
2. For each step k , we construct the formula $\Phi_k = \Phi_{\text{Edges}} \wedge \Phi_{\text{Count} \leq k}$.
3. We query the SAT solver: if Φ_k is satisfiable, we try a smaller k ; otherwise, we increase k .

Results. This exact approach provided a rigorous "Ground Truth". For small instances ($N \approx 50$), we confirmed that the Nash Equilibria found by our Strategic Games were often optimal or very close to optimal (size difference $\leq 1-2$ nodes), validating the effectiveness of the game-theoretic heuristics.

7 Conclusion and Comparative Analysis

This project successfully established a comprehensive framework to analyze Network Security through the lens of Algorithmic Game Theory. By integrating non-cooperative dynamics, cooperative stability analysis, and mechanism design, we moved from identifying theoretical security sets to simulating their economic implications.

7.1 Performance Benchmark

To quantify the trade-off between solution quality and computational cost, we benchmarked our algorithms across all topologies. Figure 1 and Table 4 summarize the execution times for increasing network sizes.

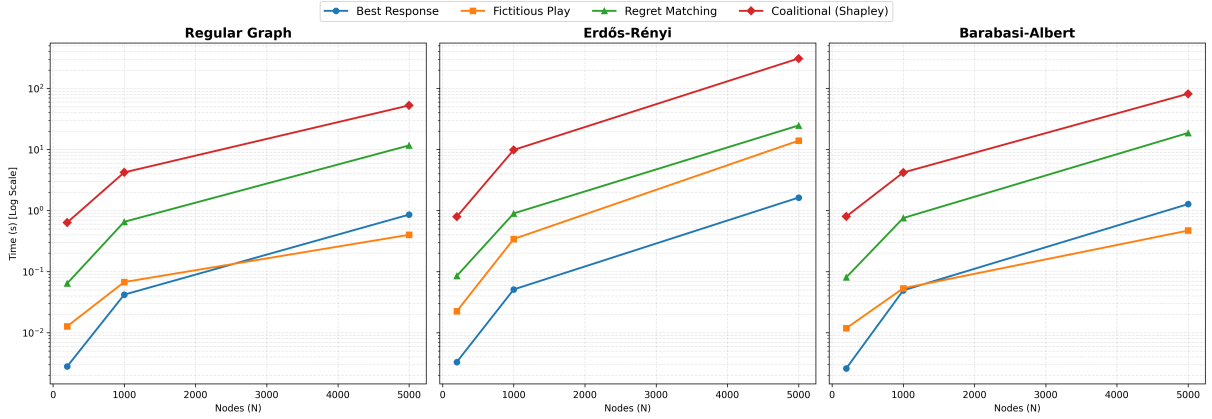


Figure 1: Runtime scalability (Log Scale). The Best Response Dynamics (Blue) scales linearly, while the Coalitional approach (Red) exhibits exponential growth, particularly on unstructured Erdős-Rényi graphs.

The results highlight a dramatic divergence in scalability:

- **Strategic Efficiency:** The *Best Response Dynamics* proved exceptionally efficient, converging in under 2 seconds even for $N = 5000$ nodes. This confirms the viability of decentralized, myopic learning for large-scale networks.
- **Coalitional Bottleneck:** The *Shapley Value* computation, despite Monte Carlo approximations, remains the computational bottleneck. On Erdős-Rényi graphs, the high variance in node degrees forced the algorithm to run for over 300 seconds to achieve convergence, making it suitable only for offline analysis of smaller critical infrastructures.

Algorithm	Execution Time (seconds) for $N = 5000$		
	Regular	Erdős-Rényi	Barabasi-Albert
Best Response	0.86	1.63	1.27
Fictitious Play	0.40	13.93	0.47
Regret Matching	11.64	24.72	18.65
Coalitional (Shapley)	52.63	309.24	81.72

Table 4: Runtime comparison for large instances ($N = 5000$). The Strategic approaches are orders of magnitude faster than the Cooperative analysis.

In summary, this work illustrates that while game-theoretic equilibria provide robust solutions for decentralized security, the underlying network topology is the primary determinant of computational feasibility, security cost, and the distribution of market power.